Reglas de seguridad

Mi Market · Cloud Firestore, Cloud Storage y clave de Google Maps

Este documento explica las reglas que protegen los datos de Mi Market en Firebase y la forma en que se restringe la clave de Google Maps. Las reglas completas están en los archivos **firestore.rules** y **storage.rules** de la raíz del proyecto; aquí se copian tal cual y se explica qué permite cada bloque.

1. Qué se protege

La app guarda cuatro tipos de información. Cada uno tiene una ruta propia y un dueño claro:

Ruta	Contenido	Quién es el dueño
productos/{productoId}	Catálogo: nombre, descripción, categoría, precio, stock, URL de la foto y el UID de quien lo creó.	El usuario indicado en **creadorUid**. Un administrador también puede editarlo.
tiendas/{tiendaId}	Locales de Mi Market con dirección, coordenadas, horario y teléfono.	Solo administradores.
usuarios/{uid}	Perfil: nombre, correo, rol, dirección de entrega y registro del consentimiento de ubicación.	El propio usuario (su UID de Firebase Auth).
usuarios/{uid}/pedidos/{pedidoId}	Historial de compras con productos, total, fecha y punto de entrega.	El propio usuario.
Storage: productos/{uid}/{archivo}	Fotos de productos comprimidas a JPEG (lado mayor de 1280 px).	El usuario cuyo UID aparece en la carpeta.

2. Matriz de permisos

La tabla resume lo que deja hacer cada regla. «Autenticado» significa que hay una sesión iniciada con correo y contraseña.

Ruta	Leer	Crear	Editar	Eliminar
productos	Cualquier usuario autenticado	Autenticado, con creadorUid igual a su UID y datos válidos	El creador o un admin, sin cambiar creadorUid	El creador o un admin
tiendas	Cualquier usuario autenticado	Solo admin	Solo admin	Solo admin
usuarios/{uid}	Solo el dueño	Solo el dueño y con rol «cliente»	Solo el dueño, sin cambiar rol ni correo	Solo el dueño

Ruta	Leer	Crear	Editar	Eliminar
pedidos	Solo el dueño	Solo el dueño y con datos válidos	Nadie (un pedido no se modifica)	Solo el dueño
Fotos en Storage	Cualquier usuario autenticado	Solo en su carpeta, imagen de menos de 5 MB	Solo en su carpeta, imagen de menos de 5 MB	El dueño de la carpeta o un admin

3. Reglas de Cloud Firestore

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    function autenticado() {
      return request.auth != null;
    }

    function esDueno(uid) {
      return autenticado() && request.auth.uid == uid;
    }

    function esAdmin() {
      return autenticado()
        && exists(/databases/$(database)/documents/usuarios/$(request.auth.uid))
        && get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data.rol == 'admin';
    }

    function productoValido() {
      let d = request.resource.data;
      return d.keys().hasOnly(['nombre', 'descripcion', 'categoria', 'precio', 'stock',
                               'imagenUrl', 'imagenRuta', 'creadorUid', 'creadoEn', 'actualizadoEn'])
        && d.nombre is string && d.nombre.size() >= 2 && d.nombre.size() <= 80
        && d.descripcion is string && d.descripcion.size() <= 500
        && d.categoria is string && d.categoria.size() > 0

        && d.precio is number && d.precio > 0 && d.precio <= 100000
        && d.stock is int && d.stock >= 0
        && d.imagenUrl is string
        && d.imagenRuta is string;
    }

    function perfilValido() {
      let d = request.resource.data;
      return d.keys().hasOnly(['nombre', 'email', 'rol', 'direccionEntrega', 'latEntrega', 'lngEntrega',
                               'aceptoPrivacidad', 'consentimientoUbicacion',
                               'fechaConsentimientoUbicacion', 'creadoEn'])
        && d.nombre is string && d.nombre.size() <= 60
        && d.email is string;
    }

    function pedidoValido() {
      let d = request.resource.data;
      return d.keys().hasOnly(['items', 'total', 'direccionEntrega', 'latEntrega', 'lngEntrega', 'estado',
'fecha'])
        && d.items is list && d.items.size() > 0 && d.items.size() <= 100
        && d.total is number && d.total > 0
        && d.estado == 'Registrado'
        && d.fecha is int;
    }

    match /productos/{productoId} {
      allow read: if autenticado();
```

```
      allow create: if autenticado()
        && request.resource.data.creadorUid == request.auth.uid
        && productoValido();
      allow update: if autenticado()
        && (resource.data.creadorUid == request.auth.uid || esAdmin())
        && request.resource.data.creadorUid == resource.data.creadorUid
        && productoValido();
      allow delete: if autenticado()
        && (resource.data.creadorUid == request.auth.uid || esAdmin());
    }

    match /tiendas/{tiendaId} {
      allow read: if autenticado();
      allow write: if esAdmin();
    }

    match /usuarios/{uid} {
      allow read, delete: if esDueno(uid);
      allow create: if esDueno(uid)
        && request.resource.data.rol == 'cliente'
        && perfilValido();
      allow update: if esDueno(uid)
        && request.resource.data.rol == resource.data.rol
        && request.resource.data.email == resource.data.email
        && perfilValido();


      match /pedidos/{pedidoId} {
        allow read, delete: if esDueno(uid);
        allow create: if esDueno(uid) && pedidoValido();
        allow update: if false;
      }
    }
  }
}
```

Funciones auxiliares

autenticado() comprueba que exista *request.auth*. **esDueno(uid)** compara el UID de la sesión con el de la ruta. **esAdmin()** lee el documento *usuarios/{uid}* de quien hace la petición y revisa que su campo *rol* sea «admin». Esa lectura cuenta como una lectura más en la facturación, por eso solo se evalúa cuando la comparación con el creador falla.

Productos

Leer el catálogo exige sesión, así que nadie puede descargarlo sin una cuenta. Al crear un producto la regla obliga a que *creadorUid* sea el UID de quien lo envía: un usuario no puede registrar productos a nombre de otro. Al editar, la regla exige que *creadorUid* no cambie, de modo que nadie se apropia de un producto ajeno.

productoValido() limita los campos permitidos con *keys().hasOnly()* y revisa los tipos: nombre de 2 a 80 caracteres, descripción de hasta 500, precio numérico mayor que 0 y hasta 100 000, stock entero no negativo. La app hace las mismas validaciones antes de enviar, pero la regla es la que manda si alguien usa otro cliente.

Tiendas

Todos los usuarios con sesión pueden ver las tiendas en el mapa. Solo un administrador puede crearlas, editarlas o borrarlas. Desde la pantalla del mapa, un administrador ve el botón «Publicar estas tiendas en Firestore» cuando la colección está vacía y la app muestra las cinco tiendas de ejemplo.

Usuarios y pedidos

Cada usuario solo puede leer y escribir su propio documento. Al registrarse, el perfil se crea con *rol* «cliente» y la regla rechaza cualquier otro valor. En las actualizaciones la regla compara el rol

y el correo nuevos con los guardados; si cambian, la escritura falla. Así un cliente no puede convertirse en administrador editando su perfil.

Los pedidos viven dentro del perfil y heredan esa protección. Se pueden crear y leer, pero no editar: un pedido confirmado queda como constancia. Se pueden borrar porque la opción «Eliminar mi cuenta» tiene que eliminar todo el historial.

4. Reglas de Cloud Storage

```
rules_version = '2';

service firebase.storage {
  match /b/{bucket}/o {

    function autenticado() {
      return request.auth != null;
    }

    function esAdmin() {
      return autenticado()
        && firestore.get(/databases/(default)/documents/usuarios/$(request.auth.uid)).data.rol == 'admin';
    }

    function imagenValida() {
      return request.resource.size < 5 * 1024 * 1024
        && request.resource.contentType.matches('image/.*');
    }

    match /productos/{uid}/{archivo} {
      allow read: if autenticado();
      allow create, update: if autenticado() && request.auth.uid == uid && imagenValida();
      allow delete: if autenticado() && (request.auth.uid == uid || esAdmin());
    }

    match /{todo=**} {

      allow read, write: if false;
    }
  }
}
```

Cada usuario sube fotos únicamente a *productos/{su UID}/*. La regla rechaza archivos de 5 MB o más y cualquier tipo de contenido que no empiece por *image/*. La app ya comprime la foto antes de subirla, por lo que en la práctica cada imagen pesa unos cientos de KB. La última regla, *match /{todo=**}*, cierra cualquier otra ruta del bucket.

Permiso entre servicios. La función *esAdmin()* de Storage consulta Firestore con *firestore.get()*. La primera vez que publiques estas reglas, la consola de Firebase te pedirá autorizar que Storage lea Firestore. Acéptalo. Si no lo haces, todo funciona igual salvo que un administrador no podrá borrar la foto antigua de un producto ajeno; la app registra ese caso en Logcat y sigue sin mostrar error.

5. Cómo se asigna un administrador

Ningún usuario puede darse el rol de administrador desde la app. Se asigna a mano en la consola:

1. Registra la cuenta en la app de forma normal.
2. En la consola de Firebase abre **Firestore Database** y entra a la colección **usuarios**.
3. Busca el documento cuyo ID coincide con el UID de la cuenta (el UID aparece en **Authentication**).
4. Cambia el campo **rol** de «cliente» a «admin» y guarda.
5. Cierra sesión en la app y vuelve a entrar. En **Mi perfil** verás la etiqueta «Administrador».

6. Clave de Google Maps y archivos sensibles

La clave de Maps se crea en Google Cloud con dos restricciones. En **Restricciones de aplicación** se elige «Apps para Android» y se agrega el paquete **com.example.retrofit** junto con la huella SHA-1 del certificado de firma. En **Restricciones de API** se marca solo «Maps SDK for Android». Con eso, aunque alguien copie la clave, no puede usarla desde otra app ni para otras APIs.

La clave no está escrita en el código. Se guarda en **local.properties** con el nombre **MAPS_API_KEY** y Gradle la inyecta en el manifiesto mediante *manifestPlaceholders*. Tanto **local.properties** como **google-services.json** figuran en el **.gitignore**, así que no se suben al repositorio.

El manifiesto declara *android:allowBackup="false"* para que las preferencias de privacidad no se copien a otros dispositivos mediante la copia de seguridad de Android.

7. Pruebas de las reglas

Estas pruebas se pueden repetir con dos cuentas en la app o con el **Simulador de reglas** de la consola (Firestore Database, pestaña Reglas, botón «Rules Playground»).

Caso	Resultado esperado	Mensaje que ve el usuario
La cuenta B intenta editar un producto creado por la cuenta A	Denegado (PERMISSION_DENIED)	No tienes permiso para realizar esta acción.
La cuenta B lee usuarios/{uid de A}/pedidos	Denegado	No tienes permiso para realizar esta acción.
Un cliente cambia su campo rol a «admin»	Denegado	No tienes permiso para realizar esta acción.
Se crea un producto con precio 0	Denegado por la regla y antes por la app	El precio debe ser mayor a 0
Se sube un PDF a productos/{uid}/	Denegado por Storage	No tienes permiso para subir esta imagen.
Un admin edita un producto ajeno	Permitido	El cambio aparece en tiempo real en el catálogo
Petición sin sesión iniciada	Denegado	Tu sesión terminó. Vuelve a iniciar sesión.

8. Mensajes de error en español

La clase **TraductorErrores** convierte los códigos de Firebase en mensajes que el usuario entiende. Los pedidos en la tarea:

Código de Firebase	Mensaje en la app
ERROR_WRONG_PASSWORD / ERROR_INVALID_CREDENTIAL	Correo o contraseña incorrectos.
ERROR_EMAIL_ALREADY_IN_USE	Ese correo ya tiene una cuenta. Inicia sesión.
ERROR_WEAK_PASSWORD	La contraseña es muy débil. Usa al menos 8 caracteres con letras y números.
ERROR_INVALID_EMAIL	El correo no tiene un formato válido.

Código de Firebase	Mensaje en la app
ERROR_TOO_MANY_REQUESTS	Demasiados intentos. Espera unos minutos e inténtalo de nuevo.
PERMISSION_DENIED (Firestore)	No tienes permiso para realizar esta acción.
UNAVAILABLE / sin red	Sin conexión a internet. Verifica tu red e inténtalo de nuevo.
Storage: bucket u objeto inexistente	El almacenamiento de imágenes no está disponible. Revisa la configuración de Storage.

En proyectos nuevos Firebase activa la protección contra enumeración de correos. Con ella, una contraseña equivocada llega como ERROR_INVALID_CREDENTIAL en lugar de ERROR_WRONG_PASSWORD; por eso los dos códigos muestran el mismo mensaje.